NoticeCheck: A Neutral Procedural-Compliance Checker for GST Show Cause Notice Proceedings

Mid-Submission Report

Team Bleed Tokens

Neill Rawani 2025202027 Tanisha Upadhayay 2025201088

Aryan 2025201017 Rajarshi Chatterjee 2025814004

LMA Group Project 2026

Code: `https://github.com/neill-nil/gst-scn-compliance-audit`

31 October 2026

1 The problem and why it matters

The everyday problem. India's GST law lets a tax officer demand unpaid tax through a fixed procedure under Sections 73 and 74 of the CGST Act, 2017:

1. a show cause notice (Form DRC-01) that states the grounds;
2. a window for the taxpayer to reply;
3. a personal hearing before any adverse decision (s.75(4));
4. a reasoned order (DRC-07) that stays within the notice (s.75(7));
5. proper service of every step (s.169).

Notices are issued at scale, partly by automated risk tools, and reach businesses that file correctly. About 80% of GST taxpayers are proprietorships: a shop owner or a small contractor with no tax team. When a notice arrives, often uploaded silently to a portal tab, such a person cannot tell whether the proceeding was handled lawfully. Was a hearing date given, or was the field left "NA"? Was the reply considered, or brushed aside in one line? Did the order demand more than the notice proposed?

Why it matters. High Courts quash such orders routinely: for an unsigned notice, a hearing fixed before the reply was due, a "cryptic" order that ignores a detailed reply, or a notice hidden in the portal's "Additional Notices" tab. Thousands of writ petitions reached High Courts while the GST appellate tribunal was not functioning. The cost of not knowing runs both ways. A taxpayer who misses a real defect pays an unlawful demand. A taxpayer told wrongly that "this notice is invalid" may ignore a real one. General chatbots do exactly this: they answer without checking the statute, missing real defects and inventing false ones.

How NoticeCheck helps. The user gives the show cause notice and the record of what happened next (reply, hearing notices, order). NoticeCheck returns a *compliance report* that lists every procedural defect it finds, not just a yes/no verdict. For each of 13 defect types (missing hearing, ignored reply, unreasoned order, demand beyond the notice, improper service, unsigned notice, time limit and so on) it gives:

- a verdict (defect, no defect, can't tell, or contested), with the evidence from the record;
- the provision behind every finding;
- it answers "can't tell yet" when a fact is missing, and says which fact;
- it flags points where courts disagree instead of deciding them;
- it hands urgent or unclear cases to a professional.

It checks procedure only and never decides whether tax is owed. It also does not predict whether a court would allow a petition: it lists what went wrong in the proceeding and why. A defect usually leads to a fresh hearing, not an escape from tax, so the same tool serves officers checking a proceeding before passing an order.

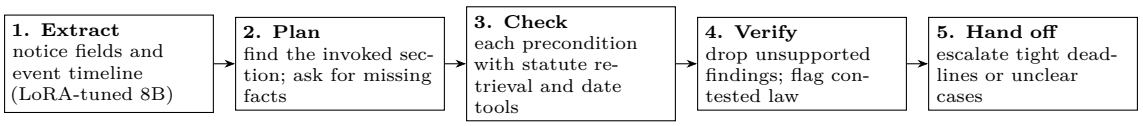


Figure 1: The planned NoticeCheck pipeline. It combines fine-tuning (1), reasoning (2–4), RAG and tool calling (3) and agentic hand-off (5). The only state-changing action (sending a reply) needs human approval.

Research questions.

- **RQ1:** Does statute-grounded, step-by-step checking find procedural defects better than the same small model prompted with the record alone, including defects that need reasoning rather than a checklist? *Falsified* if the paired F1 gain is under 10 points or its 95% interval includes zero, if its false-alarm rate exceeds the baseline's, or if it does not beat the rule checklist on reasoning-dependent defects.
- **RQ2:** Can a LoRA-tuned 8B model match a large prompted model at extracting the proceeding record?

2 Progress at a glance

Component	Status at mid-submission
Data pipeline	Done. 20,087 GST judgments screened across 24 High Courts; cleaning and paragraph segmentation; quote verifier.
Record schema 0.1	Frozen 29 Oct, as JSON Schemas plus a validator.
Draft records	425 of 426 reasoned judgments drafted by a frontier model (Gemini 3.5 Flash Lite).
Test set v1	Done. 54 real proceedings with human-verified labels, 13 reviewed twice; agreement measured.
Evaluation harness	Done. Defect precision/recall/F1, false-alarm rate, "can't tell" rate, contested rate, tool-call and unsafe-action metrics, cost, paired bootstrap.
Baselines and ablations	Done. B0 (rules), B1 (Qwen3-8B, facts only), B2 (frontier + statute), plus three ablations.
Statute corpus	Official sources registered with SHA-256 hashes; CGST Act ss.73, 74, 75, 168A, 169 extracted verbatim.
NoticeCheck system	In progress. Architecture and output contract fixed; first full runs on the test set scheduled for 1–4 Nov.
LoRA extractor (RQ2)	Planned for after the mid. Training data (non-test drafts) is ready.

3 Dataset

3.1 Source

To test a defect checker we need real proceedings where an authority has ruled, defect by defect, on whether the procedure was broken. High Court judgments provide exactly this. The judgment narrates the notice, reply, hearing and order (the *facts* the system sees), and the court then rules on each complaint (the *answer key* the system never sees). The court's overall decision is not what we predict; we use its ruling on each complaint to score the system's defect list.

Labels come only from **official High Court judgments**, taken from the AWS Open Data bucket of Indian eCourts judgments (`s3://indian-high-court-judgments`, CC-BY-4.0). It holds official PDFs with metadata for all 25 High Courts and needs no login. We chose it over the Indian Kanoon API, whose access was still pending. Kanoon and news sites are used only for discovery and cross-checking, never for labels. Every PDF is pinned by its S3 key and SHA-256 in a committed manifest, so the dataset rebuilds exactly with `scripts/fetch_data.py`.

3.2 Screening

Step (decisions 2024–2026, 24 High Courts)	Cases
Judgments with GST terms in title or opening text	20,087
− out of scope (bail, detention, registration, refunds, appeal-only, pre-GST, withdrawn, s.74A)	13,258
− duplicates (same text under another case number)	1,701
In scope: s.73/74 demand proceedings, of which:	**∼4,900**
remands and short orders (consent, "one more opportunity") that never decide the defect	3,657
tagged **reasoned** (the court rules on a procedural complaint), excluding Telangana (unreadable OCR)	426

We started with decisions from 2024–2026, where most s.73/74 procedural litigation sits; the proposal's 2019–2023 years remain available if more cases are needed. Screening uses full-text keyword rules for scope, defect cues, consent remands and duplicates. On 22 hand-checked cases the "reasoned" tag is ∼75% precise, so every test case is also confirmed by a human (§3.4). Cleaning is **deletion-only**: boilerplate lines are dropped and text is never rewritten, so every quote stays verifiable against the PDF.

3.3 What a record contains (schema 0.1)

Each case has two parts, both validated against JSON Schemas.

- **Facts**, which the checker is allowed to see:
  - provision and financial years;
  - for each notice: date, reply due date, whether a hearing date was given, the hearing date, service mode, signature, summary-only;
  - the order's date, signature and service mode;
  - a dated event timeline (scrutiny notice, DRC-01A, replies, hearings, adjournments, order, appeal);
  - what the notice, reply and order *said* (never the court's opinion of them).
- **Findings**, the answer key, hidden from every system:
  - for each procedural complaint argued: one of 13 defect types (hearing, adjournment, reply not considered, non-speaking order, demand beyond notice, service, signature, summary-only notice, limitation, extension-notification validity, consolidated notice, s.74 ingredients, other);
  - the court's finding: `defect_found`, `defect_rejected` or `not_decided`;
  - whether the point is contested;
  - the outcome.

Every value carries a verbatim quote and paragraph number, and a program checks that the quote appears in the judgment. A value without a verified quote is not used as a label. A leak guard rejects document contents that repeat a finding's words.

3.4 Building the test set: model drafts, then human review

We call the human-verified records the *gold* labels: the answer key every system is scored against.

1. **Pre-fill.** Gemini 3.5 Flash Lite drafted all 426 reasoned cases, in parallel on four free-tier keys. It makes two calls per case (facts, findings). Answers are schema-validated with one corrective retry, and every quote is checked. Result: 425 drafted, about 1 in 5 flagged for a quote or rule problem. On the 9 pilot cases the drafts got 0 of 25 stated facts wrong and all 8 `defect_found` labels right.
2. **Pick the review set.** We picked 60 cases rather than a random sample:
   - balanced by court (Madras 22, Delhi 15, others 23);
   - every case with a rejected finding included, so false alarms can be measured;
   - at most 4 of the repetitive Delhi template orders;
   - pilot and hand-labelled cases excluded (they are dev data);
   - judgments over 25 pages excluded for time.
3. **Review.** Each of the four team members reviewed 15 cases plus 4 second reviews in a purpose-built tool (`review_app/`). It shows the judgment with every quote highlighted beside the draft, checks the schema and quotes live, and requires every finding to be ticked against the judgment. Reviewers were assisted by an AI model that pointed to relevant paragraphs; every label was confirmed against the judgment text. Written rules fixed the hard calls:
   - consent or "equitable" remands are `not_decided`;

- a clear conclusion outweighs the words "prima facie";
- "disputed facts, go to appeal" is `not_decided`;
- an order set aside for the "Additional Notices" tab is a service defect.

4. **Adjudication.** Disagreements, scope questions and duplicates were settled case by case, with reasons recorded in `data/gold/adjudication.csv`. Six cases were removed: s.75(12) recoveries (×2), a pre-GST Service Tax demand, a s.161 rectification, and one Delhi common order filed under three case numbers (kept once).

Test set v1 (gold labels)	
Cases	54
Complaints argued	84
found / rejected / not decided	36 / 27 / 21
Courts	10
Madras / Delhi / other	21 / 13 / 20
Contested points (per judgment text)	10

Quality checks	
Inter-reviewer κ, findings (13 cases, 26 cells)	0.54
Outcome agreement	13/13
Fact agreement per field	89–100%
Draft facts unchanged by review	169/183 (92%)
Draft findings unchanged by review	67/90 (74%)
Records passing schema and quote checks	54/54

Table 1: The human-verified test set and its quality checks.

The draft corrections show where a large model goes wrong on judgments:

- it missed complaints (9);
- it listed complaints that were never argued (6);
- it called a point "not decided" when the court did decide it (6), usually by quoting the petitioner's argument instead of the court's answer;
- it chose a wrong type (e.g. "non-speaking order" for an ex-parte hearing complaint).

Agreement of $\kappa = 0.54$ is moderate. Most disagreements are about whether the court really decided a point.

4 Baseline experiments

Task and inputs. Every system receives the facts of a case, built from the human-verified record, with the court's findings, the outcome and its reasoning removed. For each of the 13 defect types it answers "defect", "no defect" or "can't tell", with the provision it relies on. There are two views:

- `full_record`: all facts;
- `notice_only`: only the notice. Here most defects cannot be judged, so "can't tell" is the honest answer.

Scoring. Only complaints actually argued in each case are scored, against the court's finding:

- "defect" on a `defect_found` complaint is *found*; any other answer is *missed*;
- "defect" on a `defect_rejected` complaint is a **false alarm**;
- `not_decided` is not scored.

That gives 36 real defects and 27 rejected complaints. At the mid a predicted defect counts when its *type* matches. The proposal's strict score (type *and* cited provision must match) will be added once the defect-to-provision table is frozen. Each system was run once. Confidence intervals come from a paired bootstrap over cases (5,000 resamples); the final submission uses 3 runs per configuration. Defect types are also split into *rule-detectable* (hearing, signature, service, limitation) and *reasoning-dependent* (all others, e.g. reply not considered, non-speaking order, demand beyond notice, consolidated notice).

System	Model and input	Setup
B0 rules	No model. Hearing date missing, "NA" or before the reply is due → hearing; unsigned → signature; "Additional Notices" tab → service; out of time → limitation	Deterministic
B1 small model	Qwen3-8B (4-bit), facts and defect list only. *The RQ1 comparison point: same model as NoticeCheck, without tools or retrieval.*	Kaggle 2×T4 (Ollama); temperature 0, seed 1, thinking off
B2 frontier + statute	Gemini 3.5 Flash Lite, facts plus the verbatim text of CGST Act ss.73, 74, 75, 168A, 169 (official India Code PDF, hash-checked)	Free-tier API; temperature 0

Table 2: Baselines.

5 Results

System (`full_record`)	Found	False al.	Missed	Prec.	Recall	F1	FA rate	Reasoning found
B0 rules	4	2	32	0.67	0.11	0.19	0.07	0 / 21
B1 Qwen3-8B	11	5	25	0.69	0.31	0.42	0.19	7 / 21
B2 Gemini + statute	**15**	**4**	**21**	**0.79**	**0.42**	**0.55**	**0.15**	**9 / 21**

Table 3: Defect detection on the 54 test cases (36 real defects, 27 rejected complaints). FA rate = false alarms / rejected complaints.

Paired difference (95% CI)	F1	Recall
B1 − B0	+0.23 [−0.01, +0.45]	+0.19 [+0.03, +0.38]
B2 − B1	+0.12 [+0.03, +0.23]	+0.11 [+0.03, +0.21]
B2 − B0	+0.36 [+0.12, +0.57]	+0.31 [+0.11, +0.50]

Table 4: Paired bootstrap over cases (5,000 resamples). No false-alarm difference is significant.

Defect type (number of real defects)	B0	B1	B2
Hearing, s.75(4) (12)	2	4	5
Reply not considered (9)	0	7	8
Non-speaking order (4)	0	0	1
Service, s.169 (3)	2	0	1
Consolidated notice (4)	0	0	0
Other procedural (4)	0	0	0

Table 5: Real defects found, by type.

Findings.

- **Rules are a weak floor.** The checklist catches 4 of 36 defects and none of the 21 that need reading the order.
- **The small model with facts alone (B1)** finds significantly more defects than the rules (recall +0.19), mostly "reply not considered" (7 of 9). Its F1 gain is not significant at this size, and it produces the most false alarms (5).
- **The frontier model with the statute (B2)** is best on every measure and significantly better than B1 on F1 and recall.
- **What this means for RQ1.** NoticeCheck must beat B1 (F1 0.42) by at least 10 points, with an interval excluding zero and a false-alarm rate at or below 0.19, and must beat B0's 0 of 21 reasoning-dependent defects. B2 shows that the headroom is real.

6 Ablations

NoticeCheck's own ablations (without retrieval, tools, verification or hand-off; and raw judgment text vs extracted record vs human-verified record as input) need the full system and are scheduled for the final submission (§9). At the mid we ran three ablations on the inputs, the statute and the model:

Ablation	Comparison	Result
A1. Information: all facts vs only the notice	`notice_only` view for each baseline; only "can't tell" is honest here	"Can't tell" rate: B0 35%, B1 80%, B2 98%. Defects still flagged (correct / false alarm): B0 4/2, B1 1/2, B2 4/4. B1 says "no defect" instead of "can't tell" most often for demand beyond notice, limitation and "other".
A2. Statute text: does pasting the law help?	B2 with vs without ss.73/74/75/168A/169 (same model, same facts)	Without the statute: F1 0.45, recall 0.33, 5 false alarms (with: 0.55, 0.42, 4). Difference F1 +0.09 [−0.03, +0.20], precision +0.08 [0.00, +0.20]. A consistent gain, not significant at 54 cases.
A3. Model scale: same input, small vs large model	B1 (Qwen3-8B) vs B2 without statute (Gemini), both facts only	F1 0.42 vs 0.45; difference +0.03 [−0.10, +0.18]. No measurable difference.

Table 6: Ablations at the mid, with the same scoring and bootstrap as Table 4.

What the ablations suggest. B2's significant lead over B1 (+0.12 F1) splits into a small model-scale part (+0.03, A3) and a larger statute part (+0.09, A2), though neither part is significant alone. Grounding in the statute, the core idea behind NoticeCheck's retrieval step, appears to matter more than model size. The NoticeCheck ablations in the final submission will test this on the small model itself.

Error analysis

- **Most misses are honest "can't tell"s.**
  - B2 said "can't tell" for 18 of the 21 defects it missed.
  - B1 said "no defect" for 11 of its 25 misses, asserting absence where the facts are silent. This is the failure NoticeCheck's "ask for the missing fact" step targets.
- **Missed by every system:**
  - **7 hearing defects.** The court found no proper hearing, but the extracted facts hold no hearing date or "NA": the evidence lives in the court's reasoning, which every system is correctly denied.
  - **All consolidated-notice and "other" defects.** These need legal judgement or case law, not facts. This argues for retrieval over High Court precedent, which is planned.
- **False alarms come from law that depends on the case.** In two cases the notice sat in the portal's "Additional Notices" tab, and B0, B1 and B2 all flagged service. Both courts *rejected* the complaint: the taxpayer had not denied receiving the email (Patna), and the tab argument failed on the facts (Delhi). In other Delhi cases the same fact is a defect. B2 also flagged a multi-year notice in a Delhi case where that court holds consolidation lawful; the Bombay High Court holds the opposite. These are exactly the "contested" points NoticeCheck must flag rather than decide. No baseline can express "contested": all 10 contested points in the test set went unmarked.

Cost.

- B1: ∼1.1k input and ∼2.1k output tokens per case; ∼2 min per case on a T4; 1 h 45 min for 108 calls.
- B2: ∼4.8k input tokens per case (including the statute) and ∼1k output; ∼5 s per case on the free tier.
- Pre-fill: ∼2 calls per case, within four free-tier daily quotas.

7 Challenges faced and mitigations

Challenge	What happened	Mitigation (done → planned)
Most court orders never decide the defect	3,657 of ∼4,900 in-scope orders are consent or "one more chance" remands.	Three-way labels (found / rejected / not decided); remands never used as "can't tell" labels; the "can't tell" test is built by hiding facts (A1).
Noisy screening	The "reasoned" tag is ∼75% precise. Review found s.75(12) recoveries and a pre-GST Service Tax case; one common order sat under three case numbers.	Human scope check in review; adjudication file → tighten screening rules for s.75(12) and the Finance Act; deduplicate common orders by text.
Text quality	Telangana OCR unreadable; jumbled Madras table templates; garbled Hindi fonts; the paragraph splitter merges quoted orders.	Telangana excluded; quotes verified against the whole text → table-aware extraction for Madras templates.
Ambiguous labels	"Prima facie" views, equitable remands, "contentions kept open"; $\kappa = 0.54$.	Written rules and case-by-case adjudication → revise guidelines on the disagreement cases; 40 double-reviewed cases for the final, as proposed.
Model output failures	The drafting model echoed the JSON schema, wrote malformed JSON, or wrote "2021-2022" for "2021-22".	Schema validation, corrective retry and client-side fixes; 425/426 drafted. Baseline failures scored as given, not repaired.
Compute and quotas	4 GB laptop GPU; free API tier of 500 calls/day per project; Kaggle 30 GPU-h/week.	Pre-fill split across four keys; Qwen3-8B on Kaggle T4s via Ollama; disk cache, so no call is paid twice; runs resume after interruption.
Bias and leakage	The test labels started as Gemini drafts, so B2 (Gemini) may have a home advantage. Hidden findings could leak through document contents.	Findings and reasoning removed from inputs; leak guard on contents → a leakage test (predict outcome from inputs); a non-Gemini frontier model for B2 in the final, or the advantage reported.
Facts may not carry the evidence	7 hearing defects missed by all systems: the facts lack the hearing detail.	→ Extend extraction of hearing and order-content facts; add a raw-judgment-text view as an upper bound.
Reproducibility	The input builder writes fields in a different order on each run (Python set).	Content verified identical for B1/B2 → fix the field order before the final runs. Every run records commit, model digest and data hashes.

8 Code and reproducibility

- `scripts/`: fetch and screen data, clean and segment, build the review queue and the test set.
- `src/common/llm.py`: one cached, logged model client for all roles.
- `src/prefill/`: drafting, schema and quote checks.
- `review_app/`: the review tool.
- `eval/`: scoring, agreement, reports; `src/baselines/`: inputs, B0, B1, B2; `scripts/compare_runs.py`: bootstrap.
- `data/gold/records.jsonl` and `data/splits/test.txt`; all outputs and scores in `runs/`.

Every model call is logged with tokens and seconds.

9 Timeline to the final submission

The final submission delivers what the proposal promised:

- the working NoticeCheck system, which reads a notice and its proceeding record and returns the compliance report described in §1;
- the benchmark of real proceedings;
- the fine-tuned extractor;
- the full evaluation;
- a demo.

Dates	Work	Deliverable
1–4 Nov	**NoticeCheck v1, end to end.** Input: a notice and its proceeding record. Output: the defect report (verdict per defect type, provision, evidence, missing facts, contested flag, hand-off decision). Connect the tools (limitation and date calculator, statute lookup, case-law search, record lookup) and statute retrieval. Run on the 54 test cases in both views	First answer to RQ1: NoticeCheck vs B1 on defect F1 and false alarms
1–7 Nov	**Larger test set.** Verify about 100 more proceedings by hand (target ∼150, the proposal's minimum viable size; more if time allows), including one High Court held out entirely to test generalisation; 40 cases reviewed twice; sharpen labelling rules where reviewers disagreed. Add the proposal's LLM screening step, and hand-check a sample of rejected judgments to estimate how many relevant cases screening misses	Test set v2; agreement on 40 cases; screening recall
4–9 Nov	**Ablations of NoticeCheck.** Remove retrieval, tools, verification and hand-off, one at a time. Run the checker on raw judgment text, on the extractor's output and on the human-verified record. Measure one metric per capability: • retrieval: Recall@k of the right provisions, and citation faithfulness; • reasoning: accuracy of each intermediate precondition verdict; • tools: tool-call accuracy and unsafe-action rate; • hand-off: precision and recall; • verifier: false findings removed vs true ones dropped. Also measure false "invalid" claims and missed escalations. 3 runs each	Ablation and capability tables with 95% intervals
5–10 Nov	**RQ2: fine-tuned extractor.** QLoRA fine-tune of Qwen3-8B to read a proceeding and fill the record (notice fields and event timeline), trained only on non-test cases. Compare with the untuned 8B (B3) and the large prompted model; plug it into step 1 of the pipeline	Field-level F1 table; extractor in NoticeCheck
6–10 Nov	**Synthetic stress set** from the official DRC-01 template: notices with planted defects, and fully valid notices to test for false alarms	Stress-set results, reported separately
8–11 Nov	B2 with a second frontier model (removes the Gemini home advantage); case-law retrieval so contested points can be flagged with the conflicting judgments	Final baseline table
11–13 Nov	Full evaluation on test set v2 and the held-out court (3 runs, strict and type-level scores); accuracy–cost chart; trace each wrong answer to the first component that failed; validate any LLM judge on 50 human labels	Results and failure analysis
13–15 Nov	**Demo and write-up:** a walkthrough with a simple interface on one defective and one valid notice; paper, dataset documentation, repository release	Final submission